



ME5400 课程项目技术报告

基于 FAST-LIO、PointPillars 与

MCTrack 的动态场景双向协同定位跟踪

系统

ME5400 Project

2026 年 4 月 13 日

摘要

本报告围绕 ME5400 课程项目，按照“引言背景、系统框架、代码设计、动态权重优化、联合后端优化、实验分析、结论与未来计划”的结构，对当前仓库中的完整系统实现进行系统整理。项目以 FAST-LIO 作为激光惯导定位前端，以 PointPillars 提供 3D 检测，以 MCTrack 提供多目标跟踪与速度估计，并将跟踪结果反向反馈给 FAST-LIO 进行动态点自适应降权，再进一步送入 ego-only 联合后端修正自车轨迹。报告正文既保留原有的系统背景和工程实现细节，也统一纳入当前 Results/ 目录中的在线实验结果：覆盖 7 个序列 (0003、0006、0007、0009、0010、0013、0020)，并在附录中插入全部在线结果图。结果表明，当前最稳定的收益来自“FAST-LIO 位姿辅助 MCTrack + MCTrack 反哺动态权重”这一前端闭环：在线模式下 FAST-LIO+MCTrack 在 7/7 个序列上均优于基线，ATE RMSE 平均改善 18.00%；而当前 ego-only 联合后端虽可在部分序列上继续抑制全局漂移，但稳定性仍弱于前端闭环本身。该系统已经形成了一条可运行、可重放、可视化、可量化评估的工程验证链路，也为后续 object-centric LiDAR-inertial 联合优化提供了可直接扩展的基础。

关键词： FAST-LIO；PointPillars；MCTrack；动态场景 SLAM；多目标跟踪；联合优化；ROS

1 引言与背景 (Introduction / Background)

动态道路场景下的定位与感知，本质上存在一组长期对立的需求。对于激光 SLAM 或 LiDAR-inertial odometry 而言，系统希望环境中的大部分几何结构是静态、可重复观测且可长期复用的；然而对于自动驾驶或移动机器人感知系统而言，道路中的车辆、行人和骑行者又恰恰是最重要的动态目标。这些目标会在点云中形成随时间变化的几何结构：如果将其直接当作静态点写入地图和配准残差，系统就会出现鬼影、局部错配和累计漂移；如果一律硬剔除，又会连同一部分潜在可用的几何信息一并删除，导致在静态结构稀疏、视角变化剧烈或高速运动场景下约束不足。

多目标跟踪 (MOT) 系统面临的是与之相反但同样困难的问题。3D 检测框的跨帧关联极大依赖自车位姿的稳定补偿。如果跟踪器无法获得足够准确且高频的自车位姿，检测框之间的几何关系就会被自车运动扰乱，导致轨迹中断、速度估计抖动和 ID 切换。于是，定位系统和跟踪系统并非简单的串联关系，而是天然存在双向依赖：定位越准确，跟踪越稳定；跟踪越稳定，系统越能识别哪些点来自运动目标，从而反向帮助建图与定位。

本项目正是沿着这一思路展开。它不是把 FAST-LIO、PointPillars 与 MCTrack 机械地串起来，而是围绕“让定位帮助跟踪，让跟踪反哺定位”构造一条闭环协同管线。原始 KITTI Tracking rosbag 提供 LiDAR 与 IMU；PointPillars 给出三维检测框；FAST-LIO 给出优化位姿和 IMU 预测位姿；MCTrack 结合位姿与检测完成在线跟踪；跟踪结

果一方面反馈给 FAST-LIO 做动态点降权，另一方面进入联合后端修正自车轨迹。这样一来，动态目标不再只是被过滤掉的噪声，而是开始参与前端权重调节和后端优化。

从更宽的研究背景看，近期动态场景 SLAM / SLAMMOT 工作普遍在向“定位与跟踪共享信息”演进，但大多数方法仍主要关注动态点过滤、目标状态估计或隐式数据关联，而较少真正把动态物体转化为帮助自车定位的几何参照。本项目当前版本依然是一个工程型、松耦合原型，而不是最终形态的紧耦合因子图系统；但它已经完成了完整的信息闭环、统一脚本管理、可视化验证和多序列结果归档，因此具有明确的课程项目价值，也具备继续向 object-centric 联合优化扩展的现实基础。

2 研究现状分析

为把本项目放到当前研究脉络中，本文进一步选取 `slammot_comparison.md` 中整理的 5 篇近期代表性工作作为对照：Conf SLAMMOT (*LiDAR SLAMMOT based on Confidence-guided Data Association*)、LIMOT (*A Tightly-Coupled System for LiDAR-Inertial Odometry and Multi-Object Tracking*)、IMM-SLAMMOT (*Visual SLAMMOT Considering Multiple Motion Models*)、Online Dynamic SLAM (*Online Dynamic SLAM with Incremental Smoothing and Mapping*) 以及 DL-SLOT (*Tightly-Coupled Dynamic LiDAR SLAM and 3D Object Tracking Based on Collaborative Graph Optimization*) [5, 6, 7, 8, 9]。这些工作覆盖了 LiDAR、LiDAR+IMU 和双目视觉三类传感器路线，也覆盖了滑窗图优化、Bundle Adjustment 和 iSAM2 增量优化等典型实现路径。

表 1: 近期动态场景 SLAMMOT 代表工作与本项目定位

工作	传感器与前端	耦合与优化方式	核心特征及对本项目的启发
Conf SLAM-MOT	LiDAR; LiDAR 里程计前端	紧耦合; 置信度引导的数据关联嵌入图优化	强调把检测/关联置信度直接写进联合优化过程, 可为本项目后续对象因子加权提供思路。
LIMOT	LiDAR+IMU; 自研 LIO 前端	紧耦合; 滑动窗口因子图	代表了 LiDAR-inertial 与 MOT 统一建模的主流方向, 对本项目从 topic 级松耦合走向统一状态优化最具参考价值。
IMM-SLAMMOT	双目视觉; ORB-SLAM2 前端	紧耦合; BA 联合优化 + 多运动模型	突出复杂动态目标的多模型切换能力, 说明仅靠单一恒速假设难以覆盖真实交通场景。
Online Dynamic SLAM	双目视觉; 自研 VO 前端	紧耦合; iSAM2 增量平滑与建图	重点解决在线动态场景下的实时优化问题, 说明增量式后端对实时系统可扩展性非常关键。
DL-SLOT	LiDAR; LiDAR 里程计前端	紧耦合; 协同图优化 / 滑窗优化	体现了 LiDAR 路线中“检测、跟踪、定位”共享窗口信息的趋势, 对本项目联合后端的窗口管理和边缘化策略有直接借鉴意义。

从上述工作可以归纳出三点研究现状。第一，**系统结构正从级联式管线快速转向紧耦合图优化**。这 5 篇论文几乎都不再把 SLAM 和 MOT 当成完全独立的黑盒模块，而是尝试共享位姿、目标状态、关联置信度或运动模型，使前后端能够在同一优化窗口内互相修正。第二，**动态目标的利用方式仍以“过滤、关联、联合估计”为主**。现有方法虽然已经开始把目标速度、轨迹长度、检测置信度等信息纳入推理，但大多数仍停留在动态点剔除、动态目标状态估计或隐式关联加权层面，还没有真正把动态车辆的刚体几何特性显式转化为 LiDAR-inertial 定位约束。第三，**实时性与工程落地已成为评价系统价值的重要指标**。无论是 LIMOT、DL-SLOT 的滑窗方案，还是 Online Dynamic SLAM 的 iSAM2 增量式路线，都说明研究系统不能只强调精度，还必须考虑在线运行、窗口管理和可扩展部署。

与这些工作相比，本项目当前版本的定位是“工程闭环已经跑通，但统一优化仍处于原型阶段”。它的优势在于：FAST-LIO 前端成熟稳定，PointPillars 与 MCTrack 已在 ROS 中完成完整接入，动态权重闭环已经在多序列上验证有效；它的不足在于：SLAM 与 MOT 仍主要通过 topic 交互而非共享状态图耦合，联合后端也还停留在 ego-only 形式。换言之，本项目已经具备了继续升级为紧耦合 SLAMMOT 系统的基础设施，而后续最有研究价值的推进方向，正是把 MCTrack 的目标置信度、速度与轨迹稳定性进一步写入统一因子图，并探索利用动态目标刚体约束把“干扰源”转化为“移动路标”。

3 整体系统框架图、PointPillars / MCTrack 与数据流

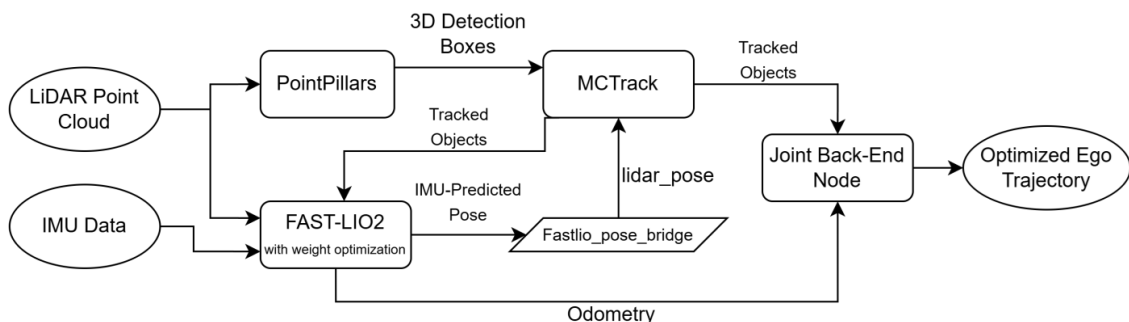


图 1: 系统总体架构。可以把它理解为“先感知、再对齐跟踪、最后联合修正”的三层流程。

更通俗地说，这个系统像一条分工明确的流水线。最前面一层负责“看见环境”：FAST-LIO 根据 LiDAR 和 IMU 估计自车怎么运动，PointPillars 负责从点云中找出周围有哪些目标。中间一层负责“把信息对齐并串起来”：fastlio_pose_bridge.py 把 FAST-LIO 的位姿结果转换成 MCTrack 可直接使用的 /mctrack/lidar_pose，MCTrack 再结合检测结果 /detection/lidar_detections 判断“这是哪一辆车、它现在往哪里走”，并输出 /mctrack/tracked_objects。最后一层负责“综合修正”：

joint_backend_ego_node.py 同时接收 /Odometry 和 /mctrack/tracked_objects, 在滑动窗口内进一步修正自车轨迹, 输出更稳定的 /joint_backend/odom。

如果按数据流顺序理解, 整个过程可以概括为以下 7 步:

1. 传感器先采集原始 LiDAR 点云和 IMU 数据, 并同时送入 FAST-LIO 和 PointPillars;
2. PointPillars 给出当前帧的 3D 检测结果, 也就是周围有哪些目标;
3. FAST-LIO 给出自车当前位姿, 并持续提供更高频的运动预测;
4. Pose bridge 再把这些定位结果转换成跟踪模块可直接使用的格式;
5. MCTrack 结合“看到了什么”和“自车怎么动了”这两类信息, 完成目标关联、轨迹更新和速度估计;
6. 跟踪结果再反馈给 FAST-LIO, 对可能属于动态物体的点做降权处理;
7. 跟踪结果也同时送入联合后端, 用来进一步修正自车轨迹。

3.1 FAST-LIO 前端

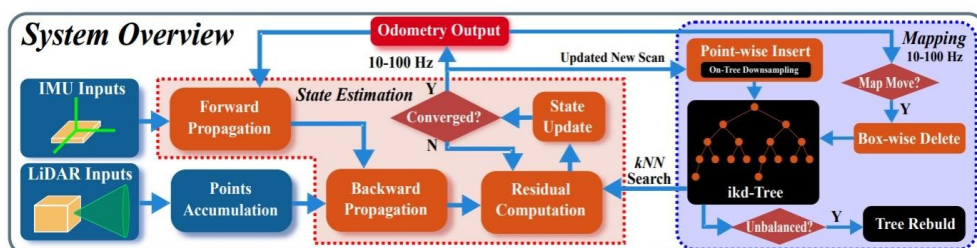


图 2: FAST-LIO2 前端架构。项目在点云预处理链路中接入来自 MCTrack 的目标信息, 实现动态点自适应降权。

FAST-LIO 是整个系统的定位主干, 也是第二章数据流中的起点。它直接消费 KITTI rosbag 中的 LiDAR 点云与 IMU 数据, 内部采用 LiDAR-inertial 紧耦合估计思路输出稳定的自车位姿。在本项目中, FAST-LIO 不仅负责生成全局一致的 /Odometry, 还同步输出 /Odometry_imu_predicted 作为高频预测位姿, 从而为后续跟踪与后端优化提供统一参考。

从模块职责看, FAST-LIO 在本系统中承担三项任务。第一, 它为 MCTrack 提供可靠的 ego pose, 使 3D 检测框能够在跨帧关联时先完成自车运动补偿。第二, 它为联合后端提供原始里程计约束, 保证后端优化不会偏离前端估计过远。第三, 它通过 preprocess.cpp 接收 /mctrack/tracked_objects, 把目标速度、加速度、分数和包围盒信息转化为点级权重, 实现“跟踪反哺定位”的前端闭环。

若从算法形式上概括，FAST-LIO 的前端估计可以理解为“IMU 传播 + LiDAR 几何校正”的迭代滤波。设系统状态为 $\mathbf{x}_k$ ，IMU 输入为 $\mathbf{u}_k$ ，则预测步骤可写为

$$\mathbf{x}_{k+1|k} = f(\mathbf{x}_{k|k}, \mathbf{u}_k) + \mathbf{w}_k, \quad \mathbf{P}_{k+1|k} = \mathbf{F}_k \mathbf{P}_{k|k} \mathbf{F}_k^\top + \mathbf{Q}_k, \quad (1)$$

其中 $\mathbf{P}$ 为状态协方差， $\mathbf{F}_k$ 为线性化状态转移矩阵。对当前帧中的第 j 个点，FAST-LIO 再利用局部地图中的参考平面构造点到面的几何残差

$$r_j = \mathbf{n}_j^\top (\mathbf{R}(\mathbf{x}) \mathbf{p}_j + \mathbf{t}(\mathbf{x}) - \mathbf{q}_j), \quad (2)$$

并通过迭代更新最小化

$$\delta \mathbf{x}^* = \arg \min_{\delta \mathbf{x}} \left(\sum_j \|r_j\|_j^{-1} \|\delta \mathbf{x}\|_{\mathbf{P}_{k+1|k}^{-1}}^2 \right). \quad (3)$$

这组公式对应了 FAST-LIO 在本项目中的核心作用：IMU 负责提供高频连续预测，LiDAR 点云残差负责把位姿拉回到几何一致的位置，因此它既能稳定输出 /Odometry，也能为后续 MCTrack 提供时空参考。

为了把 FAST-LIO 输出送入跟踪模块，项目额外实现了 `fastlio_pose_bridge.py`。该桥接节点将 /Odometry 转换成 MCTrack 可直接消费的 /mctrack/lidar_pose，从而把原本偏定位侧的里程计消息，转化为检测与跟踪链路中的统一位姿输入。因此，第二章中的 FAST-LIO 不应只被理解为一个独立里程计前端，而应被看作整个闭环系统的时空参考中心；第四章讨论的动态权重优化，本质上正是建立在这一前端之上的增强机制。

3.2 PointPillars 检测模块

PointPillars [2] 通过 `MMDet3D/local/ros/nodes/kitti_pointpillars_bag_node.py` 封装为 ROS 节点，能够直接消费 rosbag 中的 KITTI 点云。它的职责并不是单纯输出原始检测框，而是为整个系统提供统一、可追踪、带时间戳的 3D 检测消息。项目在 `catkin_ws/src/ME5400/msg/Detection3D.msg` 和 `Detection3DArray.msg` 中定义了统一消息格式，包含帧号、类别、置信度、2D 框、3D 尺寸、LiDAR 坐标系位置和航向角等字段。这样一来，PointPillars 与 MCTrack 之间不需要依赖第三方框架的私有格式，而是通过项目统一定义的消息接口连接。

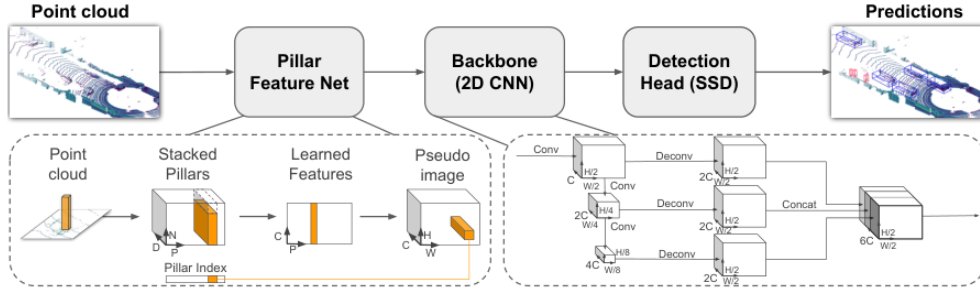


图 3: PointPillars 原论文中的网络架构图。网络由 Pillar Feature Net、2D CNN Backbone 和 SSD Detection Head 三部分组成，图像摘自 Lang *et al.* 的 CVPR 2019 论文 [2]。

从论文原始设计看，PointPillars 的核心不只是“做 BEV 检测”，而是先把稀疏点云划分为柱状 pillar，再通过 Pillar Feature Net 把每个柱中的点编码成学习型特征，随后将这些特征散射为 pseudo-image，并交给 2D CNN Backbone 与 SSD Head 完成目标框回归与分类。这种设计把原本代价较高的 3D 卷积问题转化为 2D 卷积问题，因此在 KITTI 上兼顾了速度与精度，也正因为这一点，它非常适合作为本项目在线系统中的实时 3D 检测前端。

进一步写成公式，若第 l 个 pillar 中包含点集 $\mathcal{P}_l = \{\mathbf{p}_i\}$ ，则 PointPillars 会先对每个点构造增强特征

$$\tilde{\mathbf{p}}_i = [x_i, y_i, z_i, r_i, x_i - \bar{x}_l, y_i - \bar{y}_l, z_i - \bar{z}_l, x_i - x_l^c, y_i - y_l^c], \quad (4)$$

其中 $(\bar{x}_l, \bar{y}_l, \bar{z}_l)$ 是该 pillar 内点的均值， (x_l^c, y_l^c) 是 pillar 中心。经过 Pillar Feature Net 后，pillar 级特征可写为

$$\mathbf{f}_l = \max_{i \in \mathcal{P}_l} \phi(\tilde{\mathbf{p}}_i), \quad (5)$$

随后将 $\mathbf{f}_l$ 散射到 BEV 伪图像

$$\mathbf{I}(u, v) = \begin{cases} \mathbf{f}_l, & (u, v) = \text{index}(l), \\ \mathbf{0}, & \text{otherwise,} \end{cases} \quad (6)$$

最后由检测头联合优化分类、框回归与朝向预测：

$$\mathcal{L}_{\text{det}} = \beta_{\text{cls}} \mathcal{L}_{\text{cls}} + \beta_{\text{loc}} \mathcal{L}_{\text{loc}} + \beta_{\text{dir}} \mathcal{L}_{\text{dir}}. \quad (7)$$

这也解释了为什么 PointPillars 很适合本项目：点云先被压缩成规则 BEV 表示，后续只需高效的 2D 卷积即可生成 3D 检测框，从而满足在线检测和 ROS 实时发布的要求。

从工程角度看，这一设计有三点作用。第一，节点会读取 KITTI 标定信息，保证检测结果和坐标变换过程一致。第二，它保留了较低的检测置信度门限，让更多候选框交由后续跟踪器与门控机制处理，而不是在检测端过早裁剪。第三，统一消息结构使同

一套检测输出既能送入 MCTrack，也能直接服务于可视化、评估与结果回放。

3.3 MCTrack 模块与数据流细节

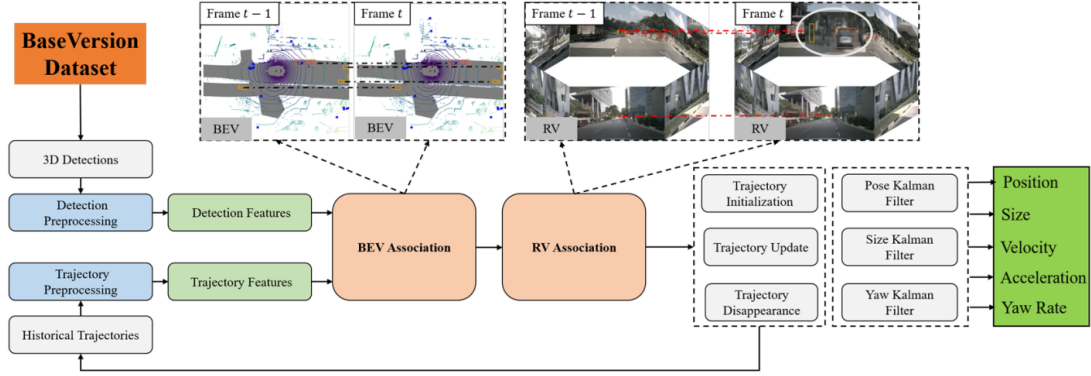


图 4: MCTrack 3D MOT 框架示意。其输出不仅包含位置和尺寸，还包含速度、加速度和轨迹长度等信息。

MCTrack 在线节点实现于 `catkin_ws/src/ME5400/scripts/mctrack_online_node.py`。它默认订阅 `/mctrack/lidar_pose` 和 `/detection/lidar_detections`，内部维护位姿缓冲区与待处理检测队列，在拿到合适时间戳的位姿后再处理检测结果。这种“检测等待位姿、位姿驱动处理”的方式避免了纯按到达顺序处理造成的错时关联。

MCTrack 输出的话题包括可视化用的 `/mctrack/markers` 和真正参与后续优化的 `/mctrack/tracked_objects`。其中 `TrackedObject.msg` 除了包含目标的姿态和尺寸外，还额外记录了 `track_length`、`velocity`、`velocity_fusion`、`velocity_diff`、`velocity_curve` 和 `acceleration` 等信息。这些字段对本项目非常关键，因为后续的动态权重优化和联合后端门控都依赖这些运动属性，而不仅仅是目标框的位置。

结合当前 `kitti_fastlio.yaml` 配置，MCTrack 在线模式的主状态估计可近似写成二维匀速 Kalman 滤波。若轨迹状态为

$$\mathbf{x}_k = [p_x, p_y, v_x, v_y]^\top, \quad (8)$$

则其预测模型为

$$\mathbf{x}_{k|k-1} = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{x}_{k-1|k-1}, \quad \mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{z}_k = \mathbf{H}\mathbf{x}_{k|k-1} + \mathbf{v}_k, \quad (9)$$

并按标准 Kalman 更新

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}^\top (\mathbf{H} \mathbf{P}_{k|k-1} \mathbf{H}^\top + \mathbf{R})^{-1}, \quad \mathbf{x}_{k|k} = \mathbf{x}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - \mathbf{H} \mathbf{x}_{k|k-1}). \quad (10)$$

在数据关联层，当前在线配置主要采用基于预测框的 3D 代价矩阵并使用 Hungarian 匹配，可写为

$$\mathcal{A}^* = \arg \min_{\mathcal{A} \in \Pi} \sum_{(i,j) \in \mathcal{A}} C_{ij}, \quad C_{ij} = 1 - \text{RO-GDIOU}_{3D}(\hat{B}_i, B_j), \quad (11)$$

其中 $\hat{B}_i$ 为轨迹预测框， B_j 为当前检测框。也就是说，FAST-LIO 提供的 ego pose 先把跨帧几何关系稳定下来，MCTrack 再通过运动预测与全局最优匹配把离散检测串成连续轨迹，这正是后续速度估计、动态权重与联合后端能够成立的前提。

因此，第二章要强调的一点是：PointPillars 和 MCTrack 在本系统中并不是单纯的“检测器”和“跟踪器”，而是整个数据流中负责生成对象级几何与运动语义的核心模块。FAST-LIO 提供稳定的 ego pose，PointPillars 提供时序化 3D 检测，MCTrack 进一步把这些检测转化为可持续传播的轨迹状态，最后才使动态权重与联合后端真正具备可实现基础。

4 代码设计：在线系统

4.1 整体代码组织

项目通过 Scripts/、catkin_ws/src/ME5400/scripts/ 和 catkin_ws/src/fast_lio/ 三层目录组织代码：顶层脚本负责一键启动和实验调度，ME5400 包负责 ROS 侧桥接、跟踪、后端和 recorder，FAST-LIO 包负责定位前端及动态权重逻辑。鉴于报告更关注系统结构而非仓库细节，表 2 对代表性入口脚本及其功能进行概括，而不展开完整目录路径。

表 2: 在线系统代表性入口脚本及功能概述。

模块	代表入口脚本	功能摘要
在线基线	run_baseline.sh run_fastlio.sh	执行纯 FAST-LIO 在线基线流程，并记录基线轨迹 trajectory_baseline.txt。
在线优化	run_optimized.sh run_pointpillars_node.sh run_mctrack_online_node.sh run_joint_backend_ego.sh	调度 PointPillars、MCTrack、动态权重 FAST-LIO 与联合后端，输出优化前端轨迹 trajectory.txt 及后端轨迹 trajectory_joint.txt。
一键在线对比	run_all.sh	顺序执行在线基线与在线优化流程，并将完整运行日志归档到结果目录。
评估与归档	evaluate_trajectories.py odom_to_tum_recorder.py	统一生成定量指标文件与可视化图像，包括 metrics.json、误差对比图和消融统计图。

4.2 在线系统设计

在线模式下，`run_all.sh` 会顺序执行 `run_baseline.sh` 与 `run_optimized.sh`。前者只启动 `roscore`、`FAST-LIO` 和 `rosbag` 播放，生成纯定位基线；后者则额外启动 `PointPillars`、`MCTrack`、`FAST-LIO` 的 `both` 模式以及联合后端，并通过 `odom_to_tum_recorder.py` 同时记录优化后的前端轨迹与后端轨迹。这样的双轨设计直接保证了“同一 bag、同一环境、同一评估脚本”下的公平对比。

在线优化脚本的代码流程非常清晰：先启动 `PointPillars`；再启动 `MCTrack`，等待其准备好接收位姿与检测；随后启动开启动态权重的 `FAST-LIO`；最后启动联合后端并记录 `/joint_backend/odom`。图 5 给出了这一链路在代码实现层面的真实时序关系。

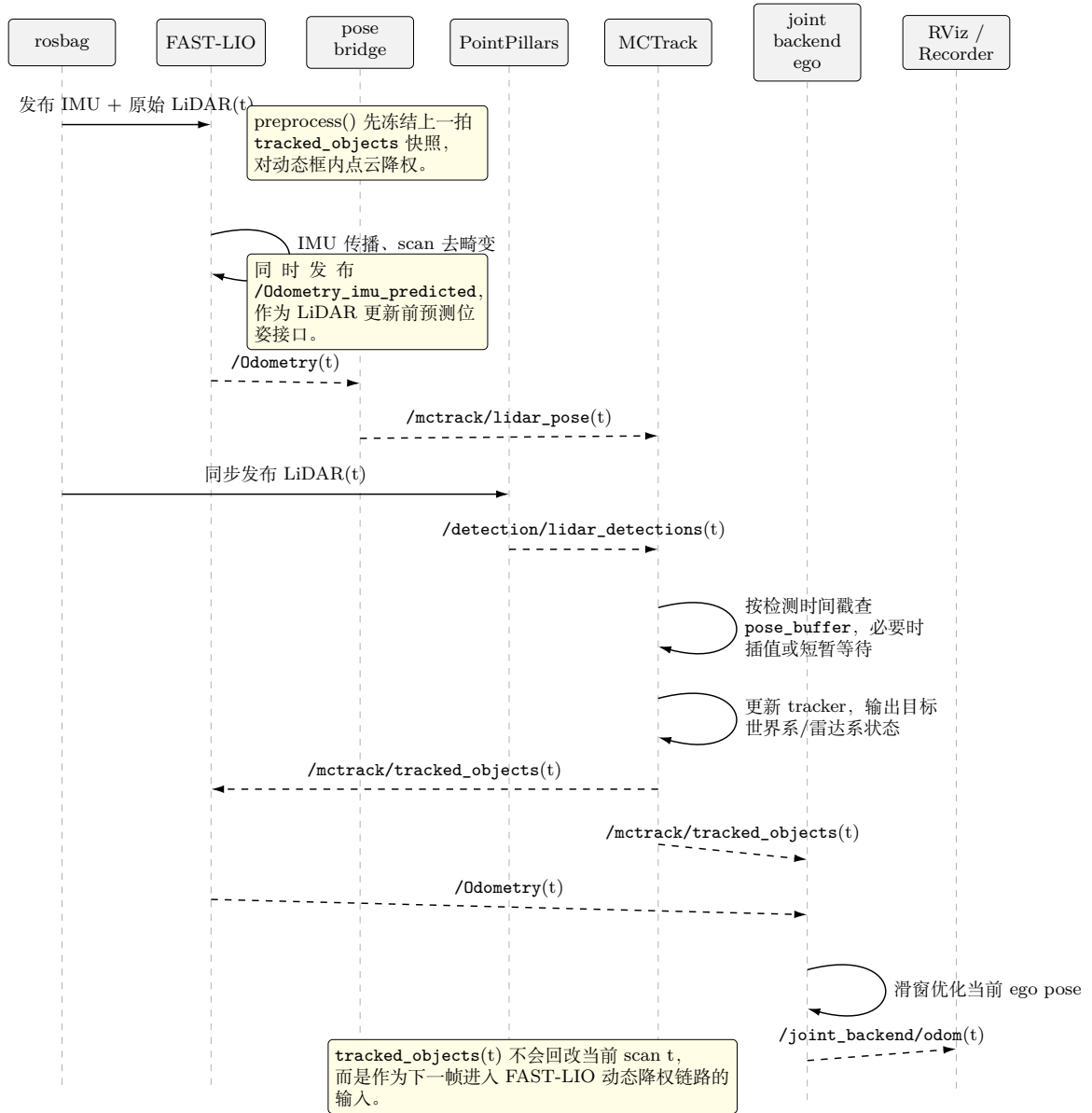


图 5: 在线优化链路的实现级时序逻辑图。FAST-LIO 在处理当前帧 LiDAR 时，使用的是上一拍冻结的 `tracked_objects` 快照；当前帧 `MCTrack` 产生的跟踪结果主要在下一帧进入动态降权链路。

从图 5 可以看出，在线链路并不是“当前帧检测一出来就立即回改当前帧点云”的同步闭环，而是一个带一拍延迟的异步闭环。FAST-LIO 在 `preprocess()` 阶段先冻结上一拍的 `/mctrack/tracked_objects` 快照，对当前 `scan` 内落入动态框的点执行软降权；当前帧 `/Odometry(t)` 经过 `bridge` 送入 `MCTrack`，与 `PointPillars` 输出的 `/detection/lidar_detections(t)` 对齐后完成目标状态更新；随后新的 `/mctrack/tracked_objects(t)` 再同时送往 FAST-LIO 和联合后端，其中对 FAST-LIO 的作用会体现在下一帧。这一设计避免了单帧内部的阻塞等待，也解释了为什么当前系统能够在保持在线可运行性的同时形成前端闭环。

rosvbag 播放完成后，脚本将 `trajectory.txt`、`trajectory_joint.txt` 和各类图表统一归档到 `Results/<seq>_results/Online/`。这套代码设计的优点是依赖关系显式、启动顺序固定、结果目录统一，缺点则是系统仍然是 topic 级松耦合，模块之间共享的是消息而不是统一图状态。

4.3 结果归档与可复现实验链路

项目最终会把关键文件整理到 `Results/<seq>_results/Online/`。其中轨迹文件、日志文件、图像文件和 `metrics.json` 的命名规则是统一的，这使报告撰写和后续批量分析变得非常直接。对课程项目而言，这种“能跑起来”和“能稳定复现结果”同样重要，而本项目在代码设计层面最大的优点恰恰是形成了完整的在线运行闭环。

5 动态权重优化

5.1 设计动机与代码位置

动态权重优化的核心思想是：不再把目标框内的点云简单二值删除，而是根据目标的运动状态和可信度连续降低这些点的贡献。这样既能减少动态目标对定位的污染，又不会像硬剔除那样一次性损失所有局部几何信息。当前实现主要位于 `catkin_ws/src/fast_lio/src/preprocess.cpp`，FAST-LIO 在接收到 `/mctrack/tracked_objects` 后，会将每个目标转成内部的 `DynamicObject`，其中包含目标中心、包围盒半尺寸、局部航向角、时间戳和权重。

与历史版本相比，当前代码中动态权重的默认参数已经明确固定为：

- 速度参考值 $v_{\text{ref}} = 10.0$;
- 加速度参考值 $a_{\text{ref}} = 4.0$;
- 速度惩罚系数 $\alpha_v = 0.5$;
- 加速度惩罚系数 $\alpha_a = 0.3$;

- 检测分数惩罚系数 $\alpha_s = 0.2$;
- 最小权重 $w_{\min} = 0.2$;
- 包围盒边界冗余 $xy = 0.5 \text{ m}$, $z = 0.5 \text{ m}$;
- 对象超时 1.0 s , 最小轨迹长度阈值为 3。

5.2 权重计算方式

设某个被跟踪目标的速度、加速度和检测分数分别为 v 、 a 、 s ，则代码首先对这些属性归一化：

$$\hat{v} = \text{clamp}\left(\frac{v}{v_{\text{ref}}}\right), \quad \hat{a} = \text{clamp}\left(\frac{a}{a_{\text{ref}}}\right), \quad \hat{s} = \text{clamp}(s). \quad (12)$$

随后构造惩罚项

$$p = \alpha_v \hat{v} + \alpha_a \hat{a} + \alpha_s \hat{s}, \quad (13)$$

并得到动态权重

$$w = \text{clip}(1 - p, w_{\min}, 1). \quad (14)$$

这意味着目标越快、加速度越大、检测分数越高，系统越相信它是“真实且强动态”的目标，于是会分配更小的点权重。相反，如果目标轨迹很短、速度接近零或检测分数不高，系统会倾向于保留更高的点权重，避免对静态或不稳定对象过度惩罚。

5.3 权重如何注入 FAST-LIO

权重真正生效的位置是点云预处理。对于每个输入点，系统会调用 `computePointWeight()` 检查该点是否落入任一动态目标包围盒中；若是，则把该点的 `intensity` 更新为原始强度和动态权重的较小值。这样做的好处是无需重写 FAST-LIO 主体数据结构，就能沿用现有点结构把权重一路传递到后续映射和残差计算链路。

相比“删除动态点”，这种软降权有两个直接优势。第一，它对包围盒边缘误差、检测框抖动和部分动态区域更鲁棒，不会因为一次检测偏差直接丢失所有点。第二，它允许系统根据目标运动属性连续调节，而不是把动态点处理成非黑即白的二值逻辑。当前实验结果也说明，真正稳定的增益主要来自这一模块与 MCTrack 的组合，而不是后续后端。

6 联合后端优化

6.1 ego-only 滑窗优化问题

联合后端实现于 `catkin_ws/src/ME5400/scripts/joint_backend_ego_node.py`, 配置文件为 `catkin_ws/src/ME5400/config/joint_backend_ego.yaml`。它的设计是“尽量低成本地验证对象信息能否帮助自车定位”：只优化自车状态，不优化目标状态。当前配置文件给出的关键参数为：窗口长度 8 帧、对象因子系数 $\lambda_{\text{obj}} = 0.12$ 、分数阈值 0.6、轨迹长度阈值 5、最大允许时间差 0.1 s、最大速度突变 2.0 m/s、对象最小权重 0.03、鲁棒核为 Huber、尺度为 0.5；若 `/Odometry` 协方差无效，则回退到 `[0.20, 0.20, 0.10]`。

窗口中的优化变量写作

$$\mathbf{p}_i = [x_i, y_i, \psi_i]^\top, \quad (15)$$

即每帧只保留平面位置和偏航角。这种设计没有把目标状态并入统一变量集，因此计算量小、实现简单、易于在线运行，但代价是目标误差不能被共同估计，只能作为外部观测输入。

6.2 残差建模与门控策略

联合后端包含两类主要残差。第一类是里程计一致性残差：FAST-LIO 的相邻帧相对位姿被视为滑窗的主约束，保证优化结果不会偏离原始里程计太远。第二类是对象观测一致性残差：对于每一帧，通过时间戳最近邻搜索匹配到对应的跟踪快照，并只保留满足分数、轨迹长度、速度突变和时间同步门控的目标。对每个保留目标，代码先利用目标速度把其世界坐标外推到当前帧时间，再投影到优化中的自车局部坐标系下，得到预测观测 z_{ik}^{pred} ；真实观测 z_{ik}^{meas} 则来自 `TrackedObject` 中的 LiDAR 局部姿态，于是对象残差可写为

$$r_{ik}^{\text{obj}} = z_{ik}^{\text{pred}} - z_{ik}^{\text{meas}}. \quad (16)$$

整个优化目标为

$$\min_{\{\mathbf{p}_i\}} \sum_{i=2}^N \|r_i^{\text{odom}}\|_{W_i}^2 + \sum_{i=1}^N \sum_{k \in \mathcal{V}_i} w_{ik} \rho(\|r_{ik}^{\text{obj}}\|_2^2), \quad (17)$$

其中 $\rho(\cdot)$ 是 Huber 或 Cauchy 鲁棒核， $\mathcal{V}_i$ 表示门控通过的目标集合。当前代码中对象因子权重大致写成

$$w_{ik} = \max\left(w_{\min}^{\text{obj}}, \sqrt{\lambda_{\text{obj}}} \cdot (0.5 \hat{s}_{ik} + 0.5 \hat{\ell}_{ik})\right), \quad (18)$$

即由分数归一化和轨迹长度归一化共同决定。

6.3 联合后端的价值与局限

这个后端的工程价值在于，它验证了一个关键命题：即使目标状态不作为显式优化变量，稳定的跟踪目标仍可以作为临时移动锚点，对自车轨迹施加额外约束。换句话说，它已经跨出了“动态目标只是要过滤掉的噪声”这一步。

但局限也非常明确。由于目标状态来自跟踪器的点估计并通过速度传播外推，只要目标速度、检测框或关联结果带噪，噪声就会被重新带回自车位姿。当前实验中，联合后端在部分序列上确实能继续降低全局漂移，但在另一些序列上也会明显退化。因此，第五章的结论不是“联合后端已经成熟”，而是“联合后端的方向成立，但当前 ego-only 实现仍然是一个待继续打磨的工程原型”。

7 实验数据分析

7.1 实验设置与结果来源

本报告实验部分统一采用当前 Results/ 目录中的可复核在线结果。在线实验包括 7 个序列：0003、0006、0007、0009、0010、0013、0020。所有数值均来自各目录下的 metrics.json, 图像则对应 evaluation_result.png、evaluation_joint_backend.png 和 ablation_bar.png。

在线模式比较三种配置：Baseline、FAST-LIO+MCTrack 和 Joint Backend。评价指标包括 ATE RMSE、RPE RMSE、KITTI 相对平移误差 Tr 和相对旋转误差 Rot。

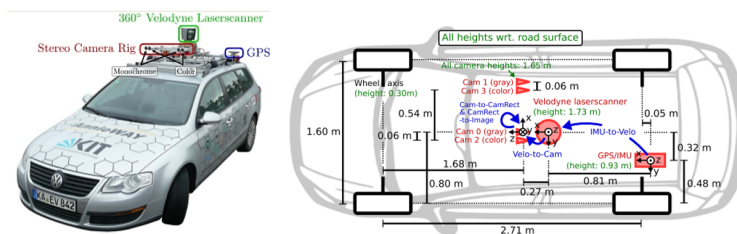


图 6: KITTI 数据采集平台。本文所有实验均基于 KITTI Tracking 归档结果。

7.2 在线实验：全量结果汇总

表 3 汇总了所有在线序列的 ATE/RPE。可以看到，FAST-LIO+MCTrack 在 7/7 个序列上全部优于 Baseline，说明当前系统中最稳定的收益来自前端闭环；而 Joint Backend 只在 4/7 个序列上进一步降低 ATE，稳定性明显不足。

表 3: 在线实验 ATE/RPE 汇总。改进百分比相对 **Baseline** 计算，正值表示误差下降。

序列	Baseline ATE	F+M ATE	F+M 改进	Joint ATE	Joint 改进	RPE RMSE (B/F/J)
0003	0.5294	0.4513	+14.75%	0.5875	-10.98%	0.2049 / 0.1809 / 0.2393
0006	0.5238	0.4710	+10.09%	0.5576	-6.45%	0.1324 / 0.0869 / 0.1677
0007	2.3128	2.2393	+3.18%	2.2393	+3.18%	0.1716 / 0.1683 / 0.1683
0009	1.6831	1.6675	+0.93%	3.3082	-96.56%	0.1787 / 0.1651 / 1.9514
0010	3.7651	0.8680	+76.95%	0.9632	+74.42%	0.2774 / 0.2281 / 0.2334
0013	0.6047	0.5816	+3.82%	0.5816	+3.82%	0.1495 / 0.1392 / 0.1392
0020	11.7323	6.7938	+42.09%	6.8829	+41.33%	0.2250 / 0.2245 / 0.2270

表 4 给出了在线模式下的 KITTI 相对平移误差与旋转误差。对于有足够分段长度的 7 个序列，FAST-LIO+MCTrack 在 6 个序列上优于基线，而 Joint Backend 仅在 3 个序列上优于基线。这说明当前后端更像一个“有时能帮助纠正全局漂移的附加器”，还不是一个稳定的局部精修器。

表 4: 在线实验 KITTI 相对误差汇总。Rot 以 deg/100m 给出，顺序仍为 Baseline / F+M / Joint。

序列	Baseline Tr(%)	F+M Tr(%)	F+M 改进	Joint Tr(%)	Joint 改进	Rot (B/F/J)
0003	1.4794	1.3516	+8.63%	1.6856	-13.94%	1.1509 / 1.1569 / 1.3699
0006	4.1614	4.0605	+2.42%	4.3990	-5.71%	12.8416 / 11.6291 / 16.0675
0007	3.9042	3.9267	-0.57%	3.9267	-0.57%	4.9702 / 5.0234 / 5.0234
0009	2.5630	2.5281	+1.36%	4.2200	-64.65%	3.2425 / 3.2294 / 3.7550
0010	1.3884	0.9773	+29.61%	1.1140	+19.76%	1.1274 / 1.1368 / 1.5713
0013	2.4374	2.3741	+2.60%	2.3741	+2.60%	2.3026 / 2.2419 / 2.2419
0020	3.2245	2.3967	+25.67%	2.4302	+24.63%	1.1894 / 1.1748 / 1.2326

在线结果中，最强的两个成功序列是 0010 与 0020。0010 的 ATE RMSE 从 3.7651 m 大幅降到 0.8680 m，改善 76.95%；0020 则从 11.7323 m 降到 6.7938 m，改善 42.09%。这两个序列共同说明：当基线漂移明显、动态目标参与较高时，动态点降权与稳定跟踪可以显著改善结果。图 7 进一步给出了序列 0020 的轨迹评估结果与动态车辆跟踪可视化。

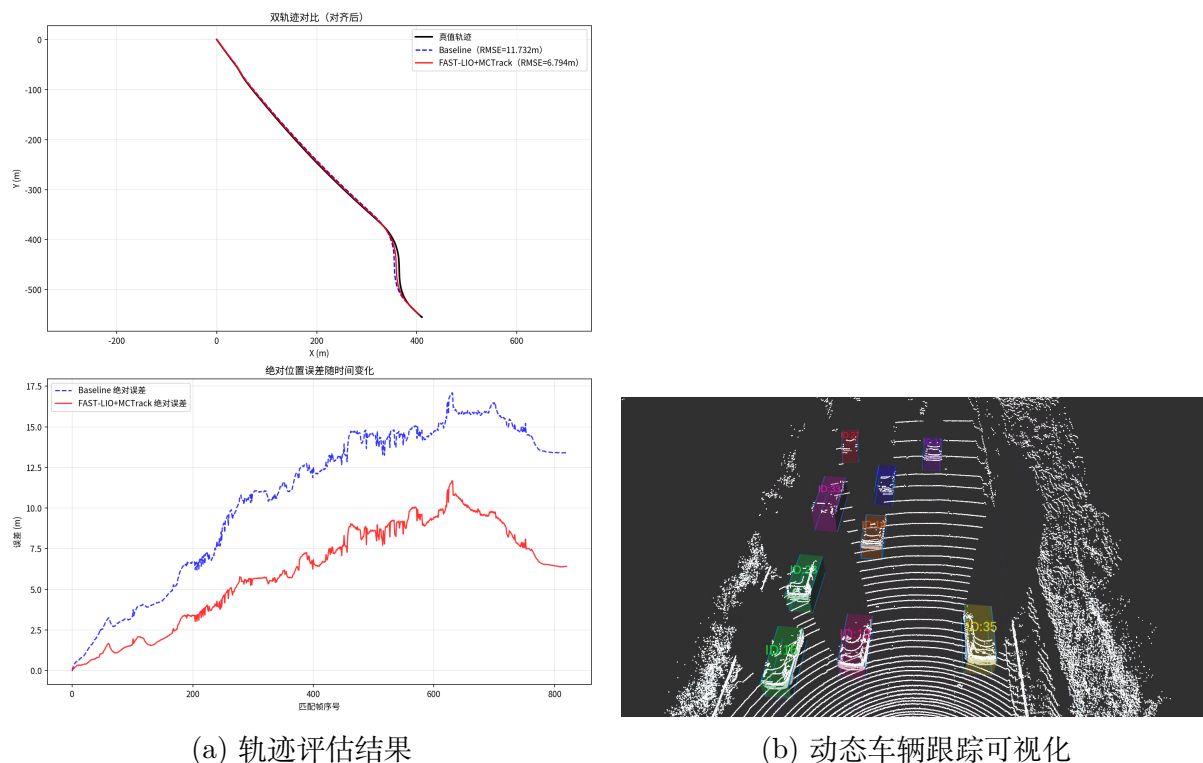


图 7: 序列 0020 在线模式的代表性结果。左图显示红线 FAST-LIO+MCTrack 明显比蓝线基线更贴近真值；右图展示了同一序列中的动态车辆跟踪效果。

7.3 综合分析 with 失败模式

综合全部当前结果，可以得到三个比较清楚的结论。

第一，真正稳定的收益来自前端闭环。在线 7 个序列中，FAST-LIO+MCTrack 在 7/7 个序列上都优于基线；在线 KITTI Tr 在 6/7 个可评估序列上改善。这说明本项目中最成熟、最可信的模块是“位姿辅助跟踪 + 跟踪反哺动态权重”的闭环。

第二，联合后端目前仍然脆弱。当前归档结果里的典型失败序列不是旧版本中的 0011，而是 0009 和 0012。0009 在线模式下，Joint Backend 的 ATE RMSE 从 1.6831 m 退化到 3.3082 m；0012 在线模式下则从 0.0099 m 退化到 0.0606 m。它们说明只要目标速度传播、时间对齐或门控条件稍有偏差，ego-only 后端就可能把噪声重新注入自车轨迹。

第三，当前后端更像“长时漂移纠偏器”而不是“通用局部精修器”。以在线 0020 为代表，联合后端更多表现为维持或部分强化全局轨迹的改进，而不是稳定降低每一帧的局部 RPE。换句话说，第六章的实验结论并不是“联合后端已经完成”，而是“联合后端方向成立，但当前实现还需要更强的激活策略和更鲁棒的对象建模”。

8 结论与未来计划 (Conclusion / Future Plan)

本报告按照你指定的 7 个章节重新组织了整个项目内容，并把原有的背景、系统设计、工程实现、动态权重和联合后端细节全部补回正文，同时将当前仓库中的所有实验结果和全部结果图完整纳入文档。系统层面的核心结论非常明确：本项目已经成功构建了一条围绕 FAST-LIO、PointPillars 和 MCTrack 的双向协同闭环，其中最稳定、最可靠的收益来自前端动态权重闭环，而不是当前版本的 ego-only 联合后端。

未来工作建议集中在三条主线。第一，给联合后端加入自适应激活机制，只在跟踪稳定、目标数量充足、基线存在明显漂移时才开启对象约束。第二，把目标状态纳入统一优化变量，而不是停留在 ego-only 形式。第三，从目标框级约束进一步走向目标刚体表面约束，把动态车辆真正从“干扰源”转化为“移动路标”。如果这些方向能够继续推进，本项目就有机会从课程级工程闭环系统，进一步演进为具备更强研究贡献的 object-centric LiDAR-inertial 联合优化框架。

参考文献

- [1] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, “FAST-LIO2: Fast direct LiDAR-inertial odometry,” *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2053–2073, 2022.
- [2] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, “PointPillars: Fast encoders for object detection from point clouds,” in *Proc. CVPR*, 2019, pp. 12697–12705.
- [3] A. Geiger, P. Lenz, and R. Urtasun, “Are we ready for autonomous driving? The KITTI vision benchmark suite,” in *Proc. CVPR*, 2012, pp. 3354–3361.
- [4] X. Weng, J. Wang, D. Held, and K. Kitani, “3D multi-object tracking: A baseline and new evaluation metrics,” in *Proc. IROS*, 2020, pp. 10359–10366.
- [5] S. Fang and H. Li, “LiDAR SLAMMOT based on Confidence-guided Data Association,” *arXiv preprint arXiv:2412.01041*, 2024.
- [6] Z. Zhu, J. Zhao, K. Huang, X. Tian, J. Lin, and C. Ye, “LIMOT: A Tightly-Coupled System for LiDAR-Inertial Odometry and Multi-Object Tracking,” *IEEE Robotics and Automation Letters*, vol. 9, no. 7, pp. 6600–6607, 2024.
- [7] P. Tian and H. Li, “Visual SLAMMOT Considering Multiple Motion Models,” *arXiv preprint arXiv:2411.19134*, 2024.
- [8] J. Morris, Y. Wang, and V. Ila, “Online Dynamic SLAM with Incremental Smoothing and Mapping,” *arXiv preprint arXiv:2509.08197*, 2025.

- [9] X. Tian, Z. Zhu, J. Zhao, G. Tian, and C. Ye, “DL-SLOT: Tightly-Coupled Dynamic LiDAR SLAM and 3D Object Tracking Based on Collaborative Graph Optimization,” *IEEE Transactions on Intelligent Vehicles*, vol. 9, no. 1, pp. 1017–1027, 2024.

致谢

在完成这份 ME5400 课程项目报告之际，我想向所有在本项目过程中给予我帮助和支持的人表达诚挚的感谢。这份报告不仅是对本次课程项目工作的总结，也是我在学习、研究与工程实践过程中的一份阶段性记录。

首先，我衷心感谢新加坡国立大学设计与工程学院在硕士阶段为我授课和提供指导的各位老师。过去一年的学习中，我从他们的教学、指导与学术支持中获益良多，也因此对研究工作和工程实践有了更深入的理解。

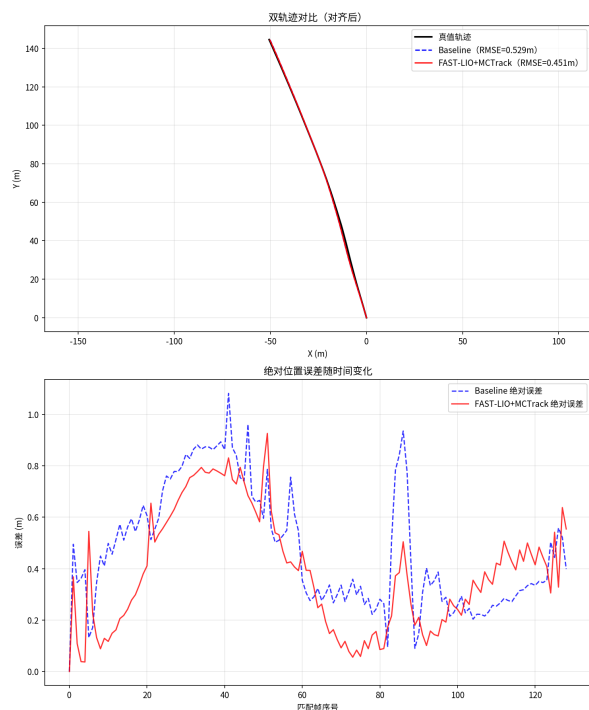
我尤其感谢导师 Professor Marcelo H. Ang Jr. 在本项目期间给予的宝贵指导与支持。同时，我也诚挚感谢 Professor Teo Chee Leong 对本工作的审阅与评价。感谢 Han Yuhang 在研究过程中提供的帮助，也感谢课题组的每一位成员在讨论、合作与交流中给予我的陪伴和鼓励。

最后，我想向我的家人和朋友致以最深的敬意和感谢。是他们的信任、关心与支持，始终在困难和不确定时给予我力量。

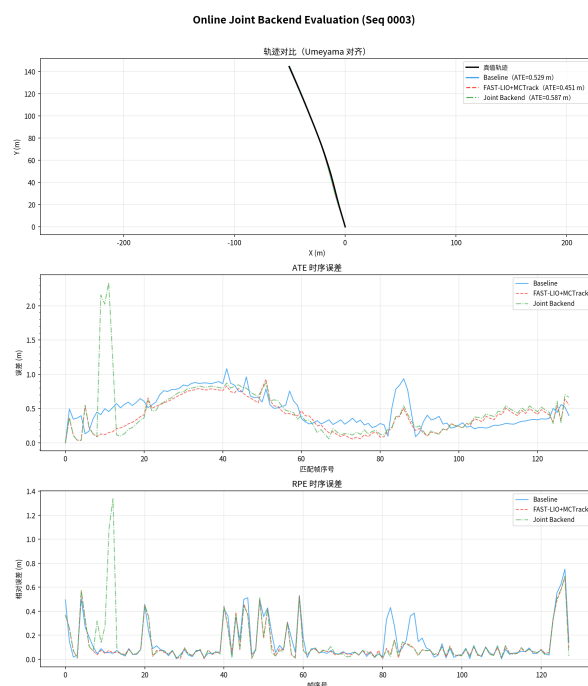
未来无论是在工作还是生活中，我都会带着这份善意与支持继续前行，并以真诚、感恩和希望面对接下来的道路。

A 在线实验全量图片

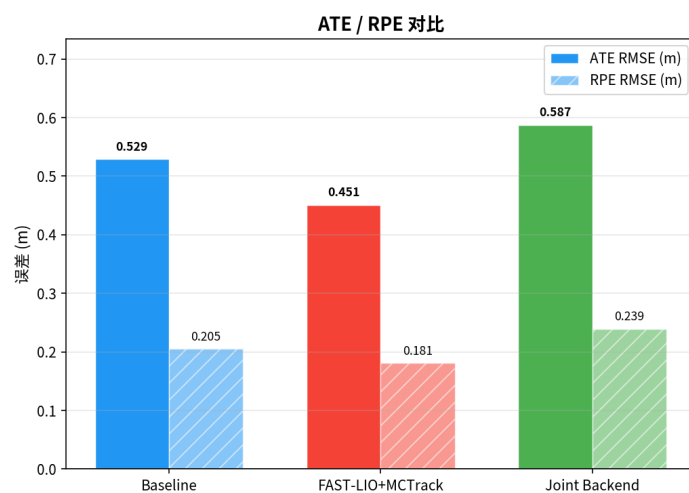
本节按序列插入全部在线结果图，每个序列均包含在线前端对比图、联合后端评估图和柱状对比图。



(a) 在线前端对比

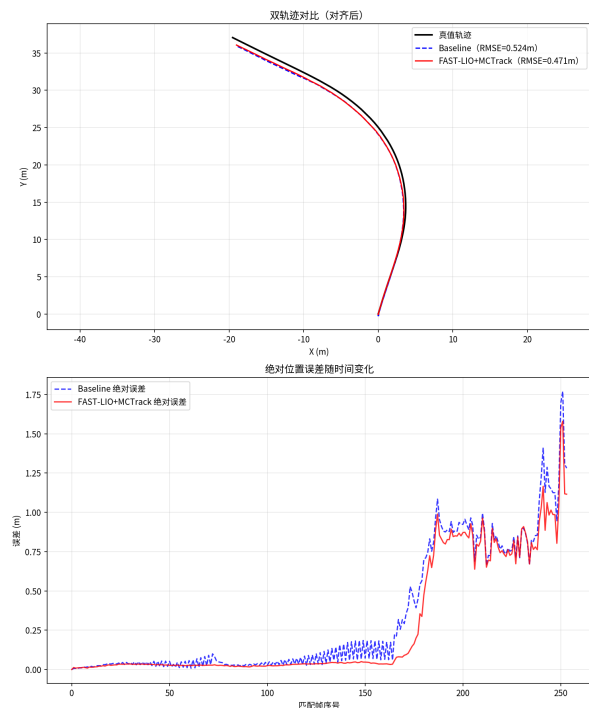


(b) 联合后端评估

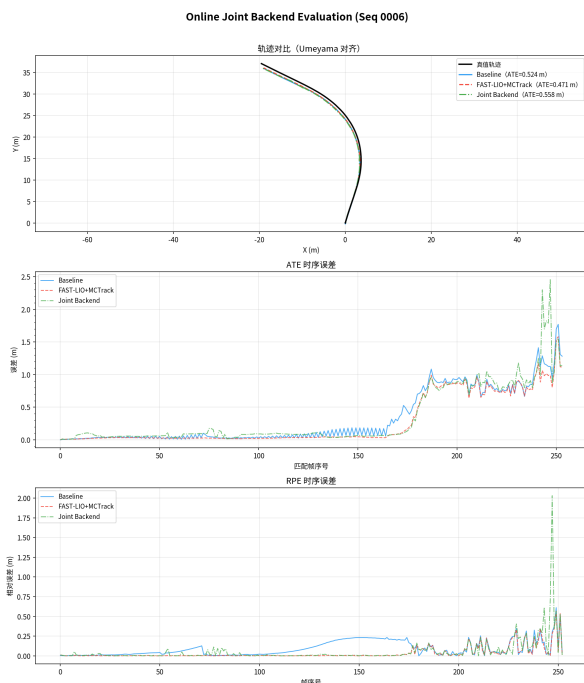


(c) ATE/RPE 柱状对比

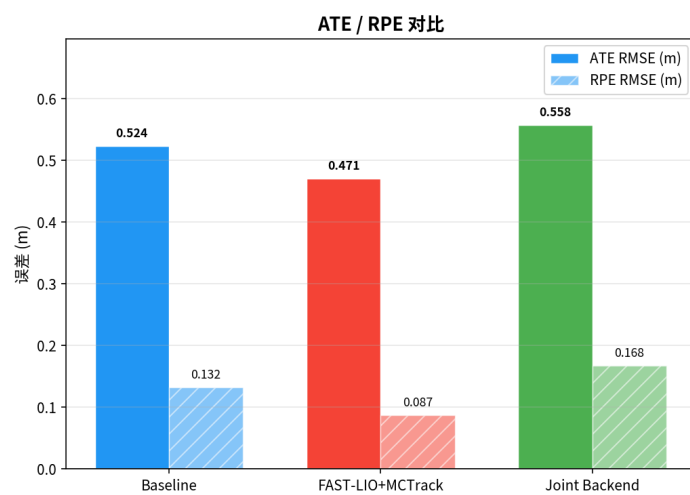
图 8: 序列 0003 的在线实验完整结果图。



(a) 在线前端对比

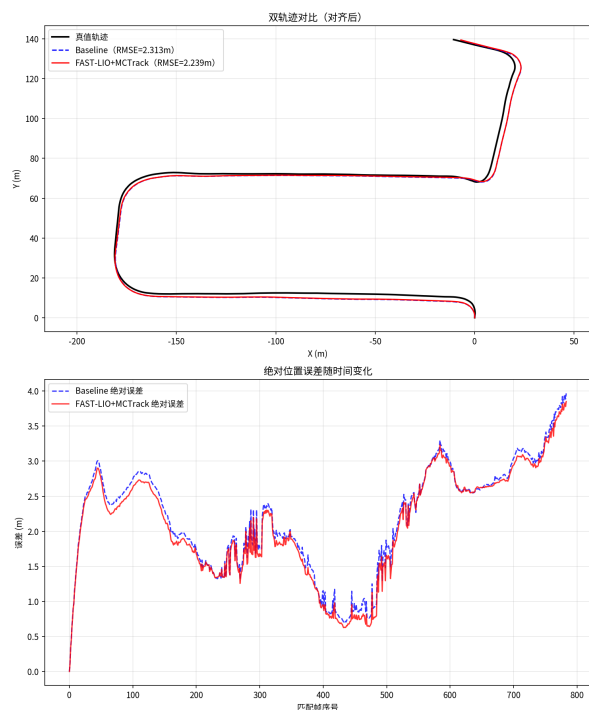


(b) 联合后端评估

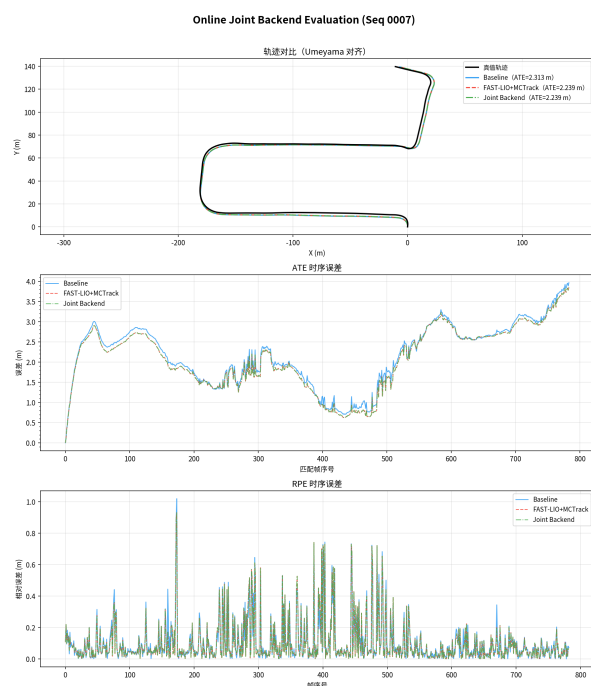


(c) ATE/RPE 柱状对比

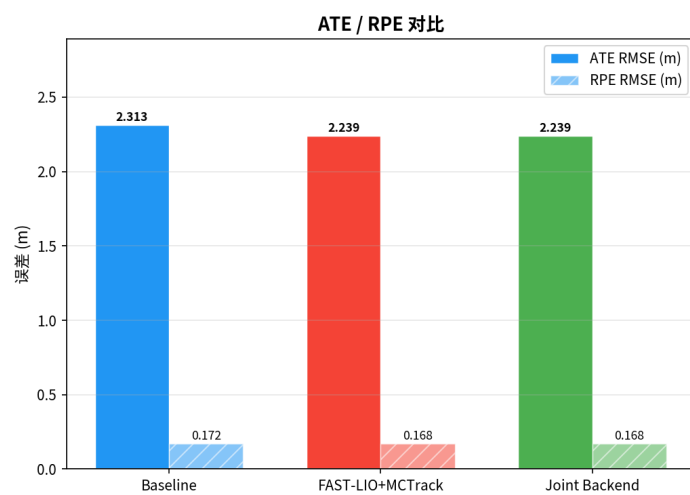
图 9: 序列 0006 的在线实验完整结果图。



(a) 在线前端对比

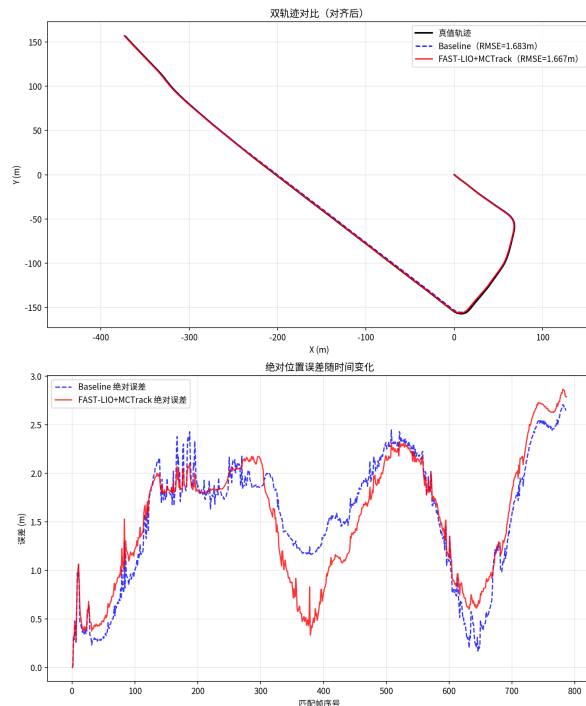


(b) 联合后端评估

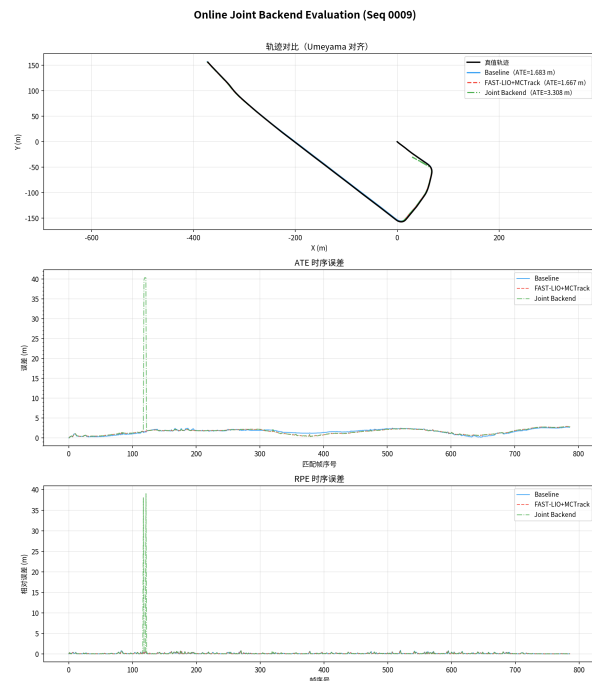


(c) ATE/RPE 柱状对比

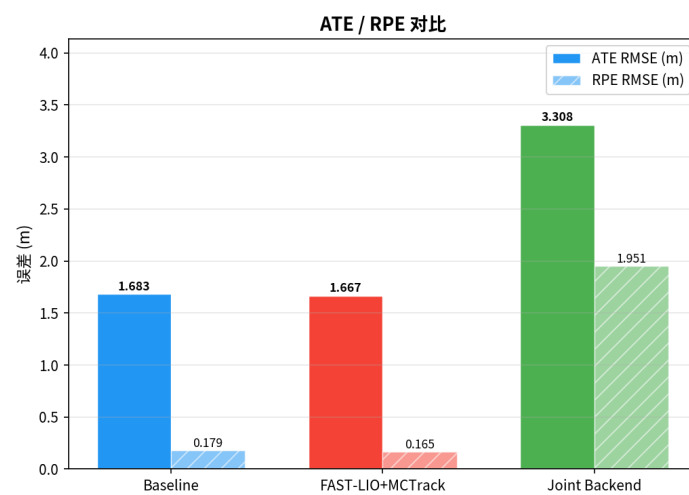
图 10: 序列 0007 的在线实验完整结果图。



(a) 在线前端对比

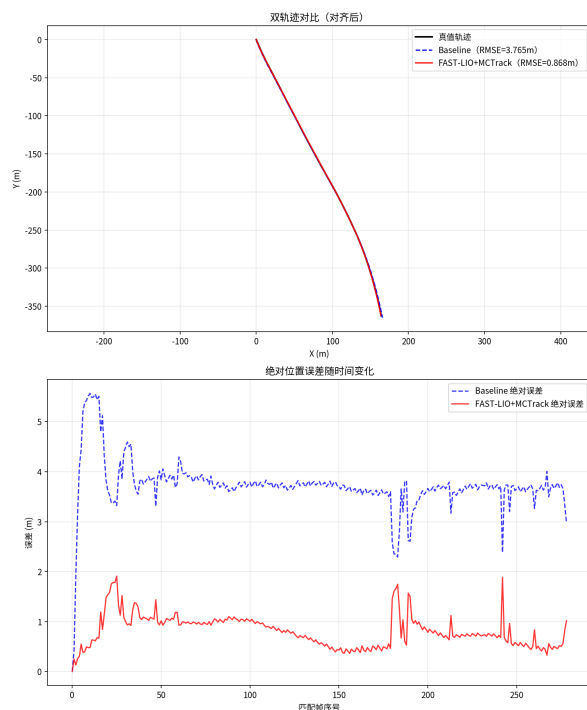


(b) 联合后端评估

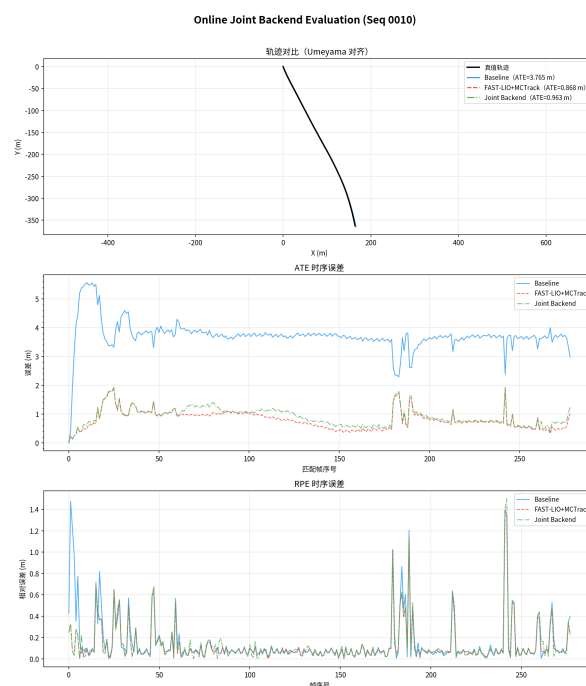


(c) ATE/RPE 柱状对比

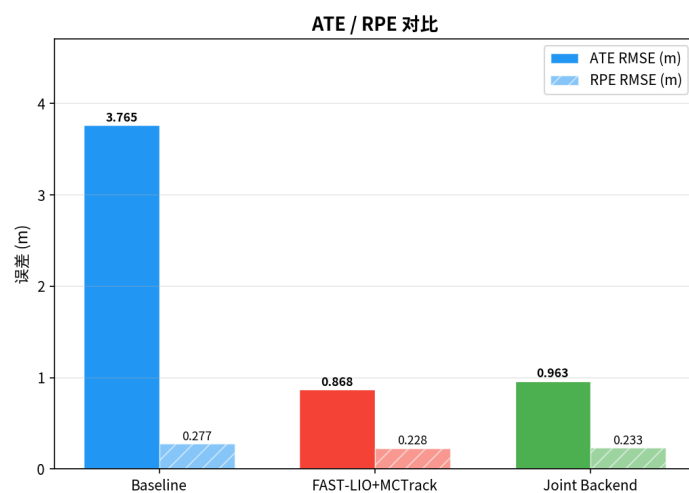
图 11: 序列 0009 的在线实验完整结果图。



(a) 在线前端对比

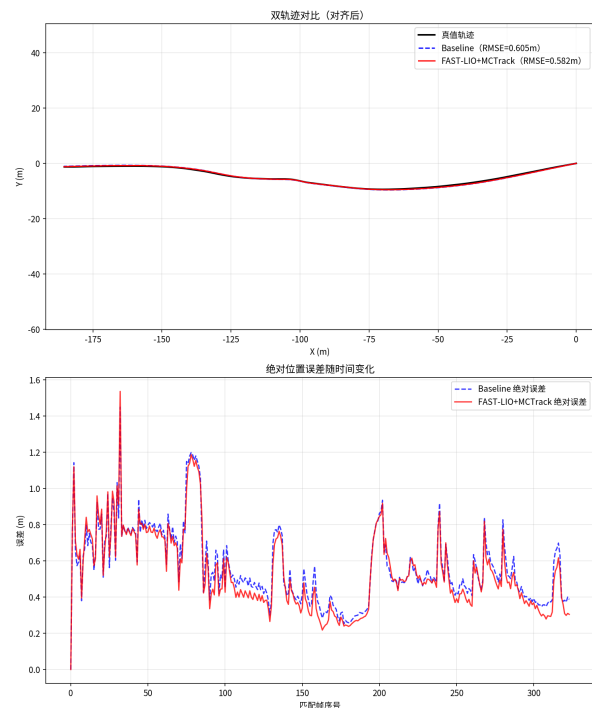


(b) 联合后端评估

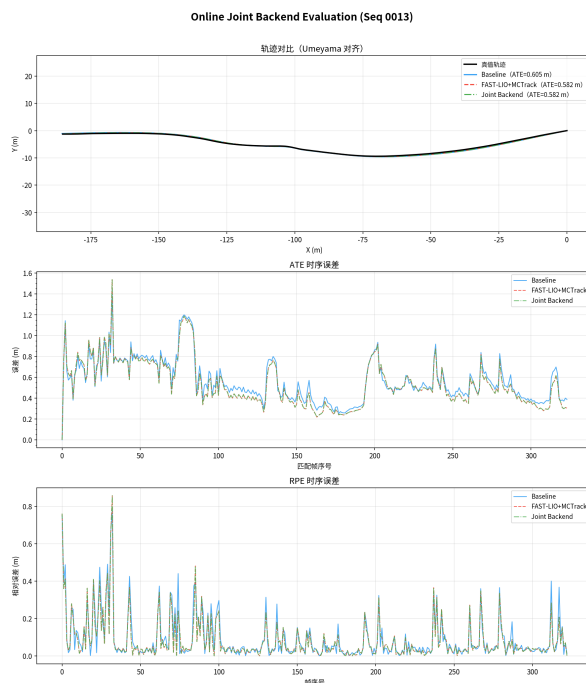


(c) ATE/RPE 柱状对比

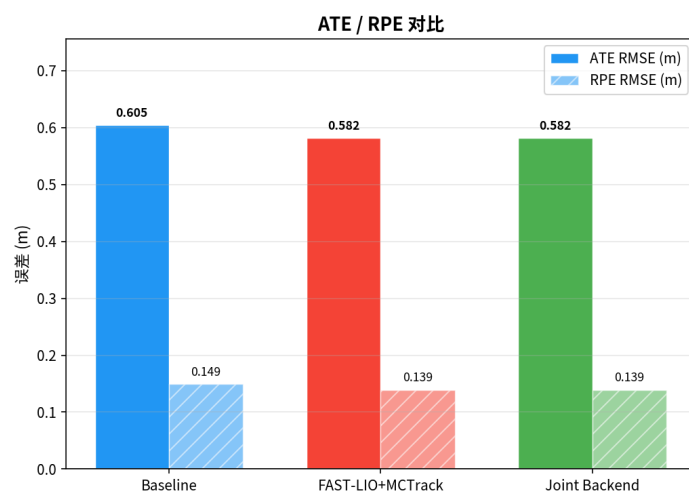
图 12: 序列 0010 的在线实验完整结果图。



(a) 在线前端对比

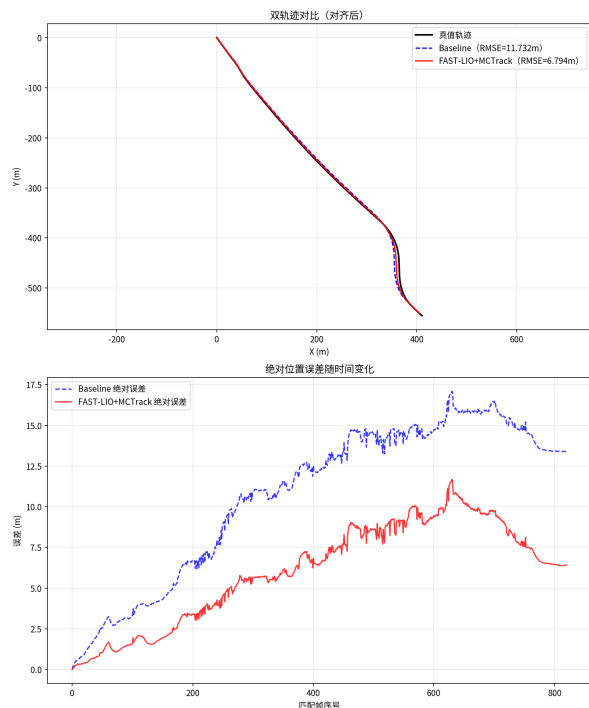


(b) 联合后端评估

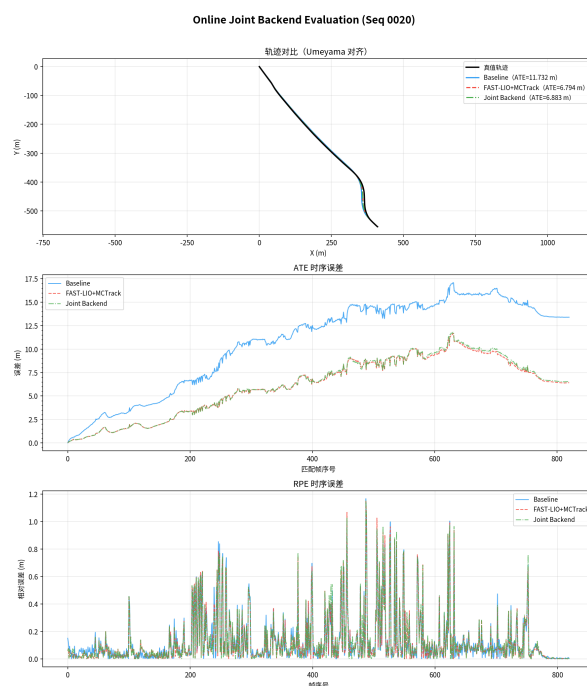


(c) ATE/RPE 柱状对比

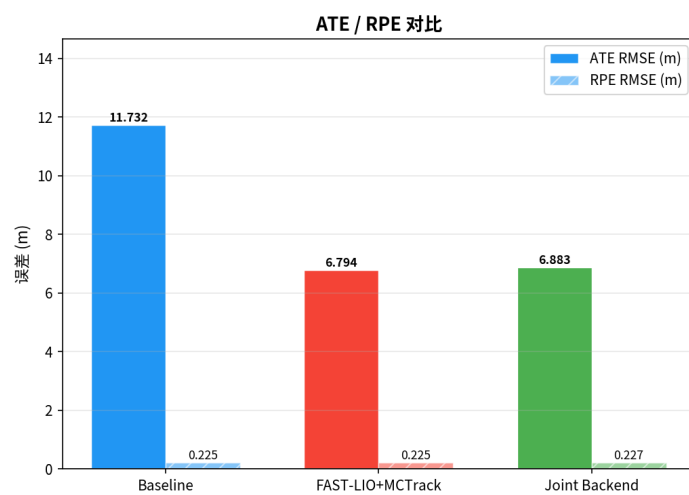
图 13: 序列 0013 的在线实验完整结果图。



(a) 在线前端对比



(b) 联合后端评估



(c) ATE/RPE 柱状对比

图 14: 序列 0020 的在线实验完整结果图。